

AI Palmistry & Tarot Intelligence Platform

Project Workflow, Architecture & Implementation Document

Item	Details
Project	AI Palmistry & Tarot Intelligence
Frontend	React.js + Vite
Backend	Python + FastAPI
Authentication	JWT-based authentication
Deployment	Vercel (Frontend) + Railway (Backend)

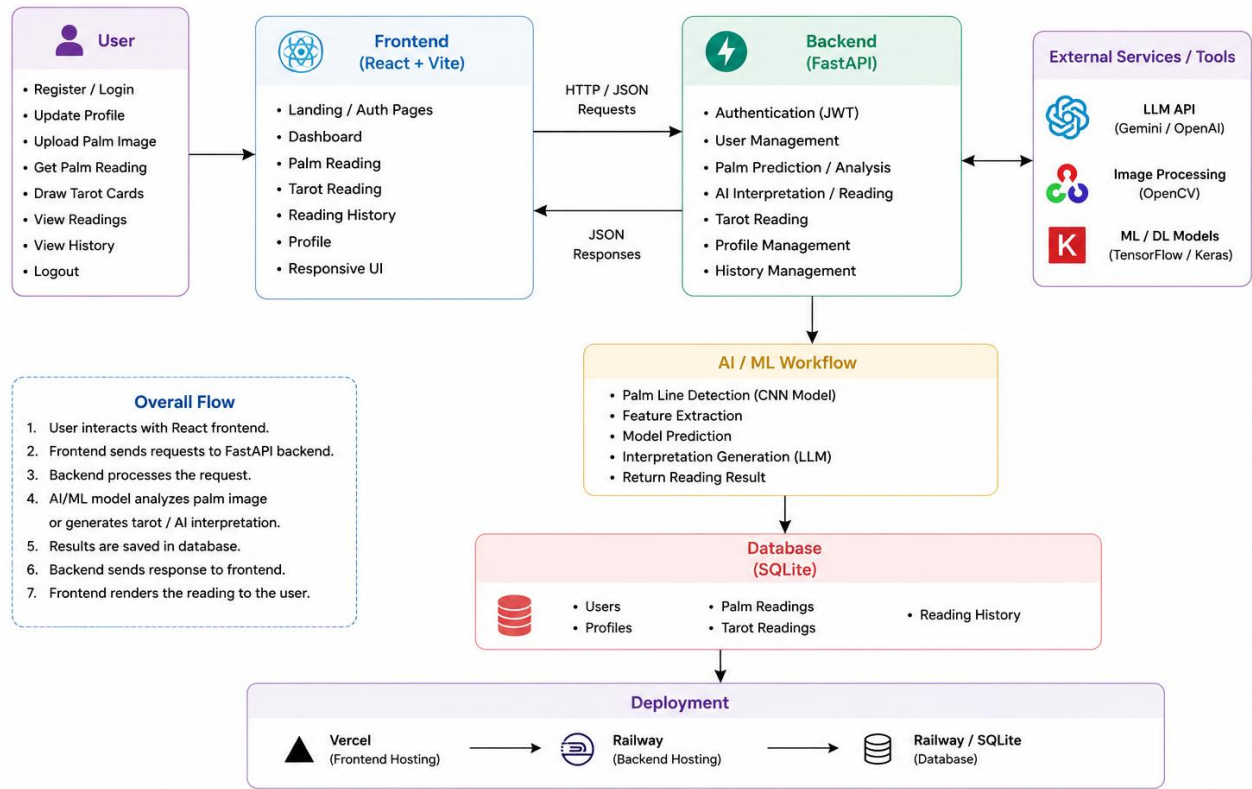
1. Project Understanding

1.1 What We're Building

The AI Palmistry & Tarot Intelligence Platform is a web application combining palm-image analysis, tarot reading, user authentication, profiles, reading history and AI-generated interpretations in one interface.

1. Allow users to register and securely log in.
2. Provide a personalized user profile.
3. Allow palm image upload and device-camera capture.
4. Send palm images to the FastAPI backend for analysis.
5. Display palm prediction and AI-generated reading results.
6. Provide an interactive tarot reading workflow.
7. Maintain reading history for authenticated users.
8. Provide a responsive interface for desktop and mobile screens.

Palmistry & Tarot Intelligence Platform – Architecture



2.1 Main Application Flow

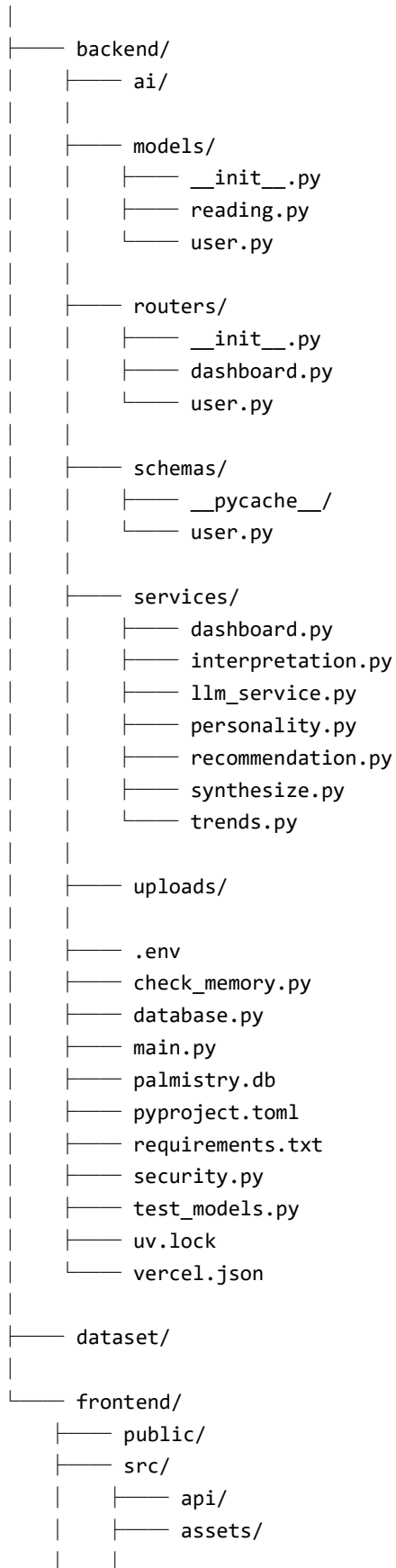
1. User opens the deployed React frontend.
2. User registers or logs in.
3. Frontend communicates with the FastAPI backend through the configured API URL.
4. Authenticated users can access Dashboard, Palm Reading, Tarot Reading, History and Profile.
5. Palm images are uploaded/captured and sent to the backend.
6. The backend processes the request and returns results.
7. The frontend presents results using responsive cards and reading sections.

8. 3. Technology Stack

Layer	Technology	Purpose
Frontend	React.js + Vite	Interactive web interface
Styling	CSS	Responsive layout and visual design
Backend	Python + FastAPI	REST API and application logic
Authentication	JWT	User authentication/authorization
AI / ML	TensorFlow / Keras + Python ML tools	Palm analysis/model workflow
Data Processing	NumPy / Pandas	Data/model processing
Database	Supabase PostgreSQL	Application data persistence
Deployment	Vercel + Railway	Frontend and backend hosting
Version Control	Git + GitHub	Source control and collaboration

4. Project Structure

Palmistry_Tarot/



```
| | | components/
| | | | DashboardHome.jsx
| | | | DashboardHome.css
| | | | PalmReading.jsx
| | | | PalmReading.css
|
| | | Profile.jsx
| | | | Profile.css
| | | | ReadingHistory.jsx
| | | | ReadingHistory.css
| | | | Sidebar.jsx
| | | | Sidebar.css
| | | | TarotReading.jsx
| | | | TarotReading.css
|
| | | pages/
| | | | Dashboard.jsx
| | | | Dashboard.css
| | | | Landing.jsx
| | | | Landing.css
| | | | Login.jsx
| | | | Login.css
| | | | Register.jsx
| | | | Register.css
|
| | | App.jsx
| | | | App.css
| | | | index.css
| | | | main.jsx
|
| | .gitignore
| | eslint.config.js
| | index.html
| | package.json
| | package-lock.json
| | README.md
| | vite.config.js
```

5. Module-by-Module Workflow

5.1 Authentication & User Management

- Registration creates a user account.
- Login validates credentials through the backend.
- JWT/token-based authentication protects authenticated functionality.
- Logout removes the client-side authentication token.
- Profile functionality allows authenticated users to manage their information.

5.2 User Dashboard

- Dashboard acts as the main authenticated landing page.
- Sidebar provides Dashboard, Palm Reading, Tarot Reading, Reading History, Profile and Logout navigation.
- Dashboard uses responsive feature cards.
- On smaller screens the sidebar changes to a hamburger-menu interaction.

5.3 Palm Reading

- User selects an image or uses camera capture.
- Image is placed into a FormData request.
- Frontend sends the image to the FastAPI prediction workflow.
- Backend returns the prediction/result.
- AI/LLM reading is retrieved/generated and displayed in the result section.

5.4 Tarot Reading

- User opens the tarot interface.
- Cards are displayed through the interactive deck workflow.
- User can shuffle/select cards.
- Card position/orientation information is displayed where implemented.
- Tarot interpretation is presented in reading cards.

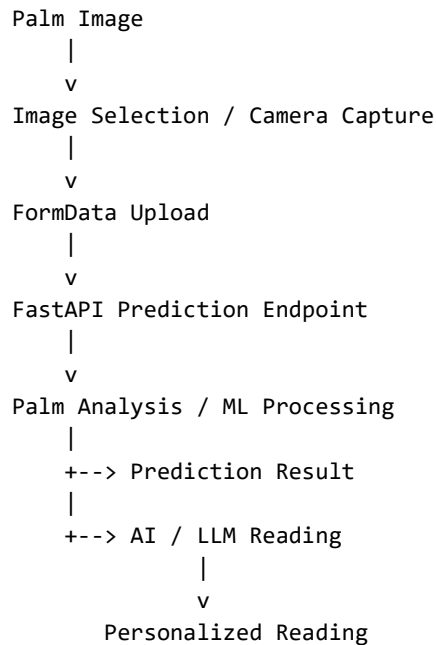
5.5 Reading History

- Authenticated users can access previous palm and tarot readings.
- History items are presented in readable cards.
- Layout is adjusted for smaller screens.

5.6 Profile

- User information is displayed in a structured form.
- Profile fields use a responsive grid.
- On small screens the fields collapse into a single column.
- User can save profile information through the existing workflow.

6. Palm Analysis Workflow



The current PalmReading frontend sends the selected image as multipart/form-data to the prediction workflow. It then displays the returned result and the AI reading.

7. Tarot Reading Workflow

9. Open Tarot Reading from the dashboard/sidebar.
10. Load the available tarot cards.
11. Shuffle the deck using the implemented workflow.
12. Select cards through the interactive interface.
13. Track card position/orientation where applicable.
14. Generate/retrieve the reading from the backend.
15. Display card details and interpretation.

8. Responsive / Mobile-Friendly UI

- Sidebar changes to a hamburger-menu style interface on smaller screens.
- Dashboard cards use responsive grid behavior.
- Login/Register cards use mobile-safe widths and spacing.
- Palm Reading upload and result containers adapt to available screen width.
- Tarot card grid adapts to smaller screens.
- Reading History cards remain within the viewport.
- Profile two-column fields collapse to a single column on smaller screens.
- Landing-page text and buttons are kept usable on narrow screens.

9. Frontend–Backend Integration

Functionality	Frontend Action	Backend Interaction
Registration	Register form	Registration endpoint
Login	Credential submission	Login endpoint / token response
Palm prediction	Upload/camera image	POST /predict (current frontend integration)
AI palm reading	After prediction	GET /llm-reading (current frontend integration)
Tarot	Shuffle/draw/read flow	Tarot API workflow
Profile	View/update profile	Profile API
History	Load previous readings	History/reading API

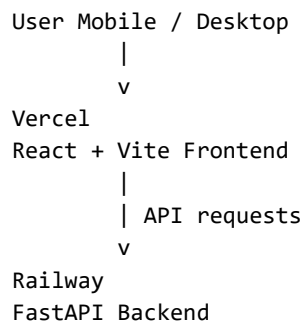
Exact route names should be checked against the final backend source before final submission.

10. Project Milestones

Milestone	Main Work
Milestone 1 – Foundation	Project setup, authentication, profile and frontend/backend integration
Milestone 2 – Palm & Tarot	Palm upload/prediction workflow, tarot interaction and related data
Milestone 3 – Interpretation	AI/LLM reading generation and personalized interpretation
Milestone 4 – UI & Deployment	Responsive UI, deployment, testing and documentation

11. Deployment

The current deployed architecture separates the frontend and backend. The React frontend is deployed on Vercel and the FastAPI backend is deployed on Railway.



- Frontend: <https://palmistry-and-tarot-intelligence-platform-hrves520t-durga11.vercel.app/>
- Backend: palmistry-and-tarotintelligence-platform-production.up.railway.app
- API Documentation: palmistry-and-tarotintelligence-platform-production.up.railway.app/docs

13. Verification Plan

16. Run frontend locally and verify it builds without errors.
17. Run backend locally and verify FastAPI starts successfully.
18. Test registration and login.
19. Verify dashboard and sidebar navigation.
20. Verify palm image upload and prediction.
21. Verify AI reading generation/display.
22. Verify tarot shuffle, selection and reading flow.
23. Verify reading history.
24. Verify profile update/save.
25. Test responsive behavior on a mobile viewport.
26. Test deployed frontend against deployed backend.
27. Check that no secrets are present before pushing to GitHub.

14. Future Scope

- Improve palm-line detection and model accuracy with a larger, better-labelled dataset.
- Expand palm feature extraction and interpretation.
- Add more tarot spreads and richer card metadata.
- Improve personalization using user history and preferences.
- Add analytics and visualization features.
- Add downloadable reading reports if required.
- Add stronger automated testing and CI/CD.
- Further optimize mobile usability and accessibility.

— End of Project Workflow Document —